

Assignment 3: Reproducible Sequence Analysis Pipeline

Answers Document

Part 2: Dynamic Programming Edit Distance

The reference sequence consists of the first 18 nucleotides of the first sequence in PM_50.fasta (accession NW_014478584.1): AAGTTAAGATAAAAAACAA. Since this sequence is also the first sequence in the file, its fragment is identical to the reference, so the resulting edit distance is 0 and the DP matrix below is symmetric with zeros on the main diagonal.

Complete Dynamic Programming matrix $M(i, j)$ for the first sequence:

Reference sequence (first 18 nt of NW_014478584.1): AAGTTAAGATAAAAAACAA

Fragment for NW_014478584.1: AAGTTAAGATAAAAAACAA

Dynamic Programming matrix $M(i, j)$:

	A	A	G	T	T	A	A	G	A	T	A	A	A	A	A	C	A	A
A	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
A	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
G	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
T	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13
T	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12
A	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11
A	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10
G	8	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9
A	9	8	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8
T	10	9	8	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7
A	11	10	9	8	7	6	5	4	3	2	1	0	1	2	3	4	5	6
A	12	11	10	9	8	7	6	5	4	3	2	1	0	1	2	3	4	5
A	13	12	11	10	9	8	7	6	5	4	3	2	1	0	1	2	3	4
A	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	1	2	3
A	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	1	2
C	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	1
A	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
A	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1

Edit distance: 0

CIGAR: MAAAAAAAAAAAAAAAAA

Wrote 50 rows to 'distances.tsv'.

Wrote 50 rows to 'alignments.tsv'.

What each cell $M(i, j)$ represents

Each cell $M(i, j)$ stores the minimum edit distance required to transform the first i characters of the reference sequence (pattern[0:i]) into the first j characters of the sequence fragment (text[0:j]). The first row ($i = 0$) represents the cost of building text[0:j] from an empty pattern (j insertions), and the first column ($j = 0$) represents the cost of deleting all i characters of pattern[0:i] to reach the empty string. The value in the bottom-right cell $M(n, m)$ is the overall edit distance between the complete reference sequence and the complete fragment.

Why Dynamic Programming avoids repeated computations

A naive recursive solution would repeatedly solve the same subproblems: computing the edit distance for pattern[0:i] and text[0:j] requires the solutions for $(i-1, j)$, $(i, j-1)$ and $(i-1, j-1)$, and these smaller subproblems are shared by many different recursive calls higher up in the recursion tree. Without memoization, the same (i, j) pair is recomputed an exponential number of times. The bottom-up tabulation approach instead computes each subproblem exactly once, storing the result in the matrix M , and reuses these stored values when computing larger subproblems, eliminating all redundant work.

Why the time complexity is $O(n \cdot m)$

The algorithm fills every cell of an $(n+1) \times (m+1)$ matrix exactly once. Computing the value of each individual cell $M(i, j)$ requires only a constant number of operations: looking up the three neighboring cells (diagonal, upward and left) and taking a minimum, plus a constant-time comparison of two characters. Since there are $(n+1)(m+1)$

cells and each one takes $O(1)$ time, the total running time is $O(n \cdot m)$.

Part 4: Regular Expression Analysis of CIGAR Strings

Why Regular Expressions are useful for CIGAR strings and bioinformatics formats

CIGAR strings, FASTA headers, GFF/VCF records and many other bioinformatics file formats follow strict, well-defined textual patterns built from a small alphabet of symbols (e.g. M, I, D, X for CIGAR operations, or fixed delimiters and field structures for other formats). Regular expressions provide a compact and declarative way to describe these patterns and to search, extract or count occurrences of specific elements without writing long, error-prone character-by-character parsing code. For CIGAR strings specifically, functions such as `re.findall()` let us count how many times each operation symbol (M, I, D, X) appears with a single line of code, which is exactly what is needed to compute alignment statistics. More generally, regular expressions make it easy to validate file formats, extract identifiers such as accession numbers from headers, and parse structured fields from semi-structured text, which is extremely common when working with genomic data formats.

Summary report (report.txt)

Sequence with smallest edit distance:

NW_014478584.1

Distance: 0

Matches: 18

Insertions: 0

Deletions: 0

Substitutions: 0

Sequence with largest edit distance:

NW_0144785128.1

Distance: 7

Matches: 13

Insertions: 2

Deletions: 2

Substitutions: 3

Sequence with largest number of matches:

NW_014478584.1

Distance: 0

Matches: 18

Insertions: 0

Deletions: 0

Substitutions: 0

Part 5: Conda Environment

1. Command to create the environment

```
conda create -n assignment3-seqanalysis python=3.12 snakemake graphviz -c  
conda-forge -c bioconda
```

2. Command to activate the environment

```
conda activate assignment3-seqanalysis
```

3. Command to export the environment

```
conda env export -n assignment3-seqanalysis > environment.yml (or conda env export  
--from-history for a more portable, minimal file).
```

4. Why an environment file is useful for reproducibility

An environment.yml file records the exact names and versions of all packages used to run the workflow. Anyone can use this file to recreate an identical software configuration on a different machine, at a different point in time, avoiding problems caused by missing dependencies, incompatible library versions, or differences between operating systems. This guarantees that the analysis can be re-executed and produces the same results regardless of where or when it is run.

Packages included and why they are required

The environment includes Python 3.12, which is required to run parse_fasta.py, alignment.py and cigar_stats.py (all written in pure Python 3 using only the standard library, specifically the 're' module for regular expressions). It also includes Snakemake, which is required to define and execute the workflow described in the Snakefile, and graphviz, which provides the 'dot' command used to render the workflow DAG as dag.png.

How another user can recreate the environment

Another user can run `conda env create -f environment.yml` to create a new conda environment with exactly the same name and package versions specified in the file, and then activate it with `conda activate assignment3-seqanalysis` before running the Snakemake workflow.

Part 6: Workflow Automation with Snakemake

Output of 'snakemake -n' (dry run)

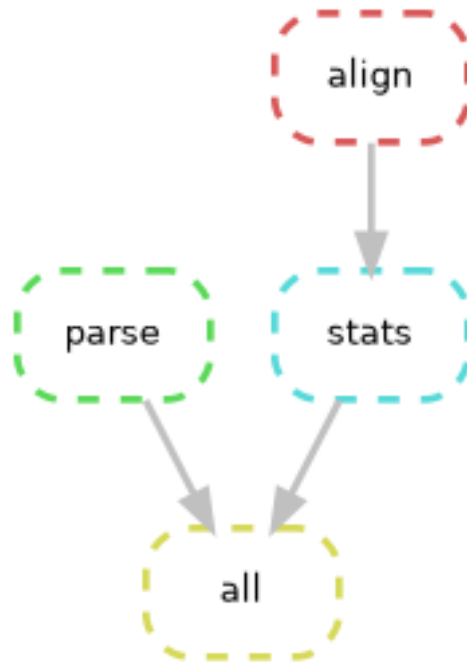
```
SNAKEMAKE
=====
Date: 2026-06-12 21:18:08
Workflow ID: 00e8d606-31f6-4946-a0b0-7370401040bf
Platform: Linux-6.18.5-x86_64-with-glibc2.39
Host: vm
User: root
Snakemake version: 9.23.0
Python version: 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
Command: /usr/local/bin/snakemake -n
Snakefile: /home/claude/assignment3/Snakefile
Base directory: /home/claude/assignment3
Run directory: /home/claude/assignment3
Working directory: /home/claude/assignment3
Config file(s): []
Config MD5: 99914b932bd37a50b983c5e7c90ae93b

Building DAG of jobs...
Job stats:
job      count
-----
align      1
parse      1
stats      1
all        1
total      4

[Fri Jun 12 21:18:08 2026]
rule parse:
  input: PM_50.fasta, parse_fasta.py
  output: sequences.tsv
  jobid: 1
  reason: Missing output files: sequences.tsv
  resources: tmpdir=/tmp
[Fri Jun 12 21:18:08 2026]
rule align:
  input: PM_50.fasta, alignment.py
  output: distances.tsv, alignments.tsv
  jobid: 3
  reason: Missing output files: alignments.tsv, distances.tsv
  resources: tmpdir=/tmp
[Fri Jun 12 21:18:08 2026]
rule stats:
  input: alignments.tsv, distances.tsv, cigar_stats.py
  output: cigar_stats.tsv, report.txt
  jobid: 2
  reason: Missing output files: report.txt; Input files updated by another job: alignments.tsv, distances.tsv
  resources: tmpdir=/tmp
[Fri Jun 12 21:18:08 2026]
rule all:
  input: sequences.tsv, report.txt
  jobid: 0
  reason: Input files updated by another job: report.txt, sequences.tsv
  resources: tmpdir=/tmp
Job stats:
job      count
-----
align      1
parse      1
stats      1
all        1
total      4

Reasons:
(check individual jobs above for details)
input files updated by another job:
  all, stats
output files have to be generated:
  align, parse, stats
This was a dry-run (flag -n). The order of jobs does not reflect the order of execution.
```

Workflow DAG



Explanation of each rule

rule parse takes PM_50.fasta as input and runs parse_fasta.py to extract, for every sequence, its accession identifier (using a regular expression on the header line) and its total length, producing sequences.tsv.

rule align takes PM_50.fasta as input and runs alignment.py, which extracts the reference sequence (first 18 nucleotides of the first sequence), computes the bottom-up Dynamic Programming edit-distance matrix for every sequence's first 18 nucleotides against this reference, performs a traceback to recover an expanded CIGAR string, and writes distances.tsv and alignments.tsv.

rule stats takes alignments.tsv and distances.tsv as input and runs cigar_stats.py, which uses regular expressions (re.findall) to count the M, I, D and X operations in each CIGAR string (cigar_stats.tsv), and then combines this information with the edit distances to generate report.txt, summarizing the sequences with the smallest and largest edit distances and the largest number of matches.

rule all defines the final targets of the workflow (sequences.tsv and report.txt), so that running 'snakemake --cores 1' triggers execution of all the rules above in the correct dependency order.

Why Snakemake improves reproducibility

Snakemake explicitly encodes the dependencies between input and output files for every step of the analysis, so the entire pipeline can be re-executed with a single command (snakemake --cores 1) without the user needing to remember the correct order of scripts or manually track which outputs are out of date. If the input FASTA file or any script changes, Snakemake automatically determines which rules need to be re-run based on file timestamps, ensuring all outputs stay consistent with their inputs. This removes the error-prone manual bookkeeping required when running each Python script by hand, and guarantees that anyone running the same Snakefile on the same input data will obtain the same outputs through the same sequence of steps.

Reproducibility Benefits of Conda and Snakemake

Conda and Snakemake address two complementary aspects of reproducibility. Conda ensures that the software environment itself (the Python interpreter, Snakemake, and any other required packages with specific versions) can be exactly recreated on any machine via `environment.yml`, eliminating issues caused by missing or mismatched dependencies. Snakemake ensures that the sequence of computational steps applied to the data is itself reproducible: the Snakefile is a precise, executable specification of how each output file is derived from its inputs, so the same input FASTA file will always produce the same `sequences.tsv`, `distances.tsv`, `alignments.tsv`, `cigar_stats.tsv` and `report.txt` through the same chain of commands. Together, a fixed software environment plus an explicit, automatically-executable workflow definition allow another researcher (or the same researcher at a later date) to obtain identical results from the same starting data.